Control-C handling in Python and Trio

Wed 19 April 2017

Trio is a new asynchronous I/O library for Python, with a focus on usability and correctness – the goal is to make it easy to get things right.

One thing well-behaved programs should do is exit cleanly when the user hits control-C. In Python this mostly Just Works without developers having to think about it too much, and as part of trio's focus on *usability*, we'd like to carry that over: there are few things more annoying than a program that refuses to quit when you hit control-C! But preserving *correctness* in the face of an interrupt that can happen at literally any moment is not at all trivial. This is a place where trio's two goals interact in a surprisingly interesting way! In this post I'll explore some different options for handling control-C, and explain Trio's solution – with a bonus deep dive into signal handling and some rather obscure corners of CPython's guts.

The tl;dr is: if you're writing a program using trio, then control-C should generally Just Work the way you expect from regular Python, i.e., it will raise `KeyboardInterrupt` somewhere in your code, and this exception then propagates out to unwind your stack, run cleanup handlers, and eventually exit the program. You don't need this article to use trio; you can start with our tutorial and be happy and productive without thinking about control-C ever again. In fact, most developers probably won't even realize that there's anything special happening at all. But if you're curious about how we make the magic go, then read on...

Contents:

- The precedent: control-C in regular Python
  - Option 1: KeyboardInterrupt
  - Option 2: a custom signal handler
- The dream
- Prior art
  - Twisted
  - Other async libraries
  - How does the Python interpreter pull it off?
- How Trio handles control-C
  - How do we know which code should be protected?
  - How do we deliver a KeyboardInterrupt if we can't raise it?
  - What if you want a manual control-C handler?
- Limitations and potential improvements
  - Issues with handing off from the C-level handler to the Python-level handler
  - Issues with the interaction between KeyboardInterrupt and `with` blocks
  - yield from and await aren't signal-safe
  - What about PyPy?
- Conclusion
- Comments

The precedent: control-C in regular Python

Before we get into event loops and all that, let's review how things work in regular Python. When you're writing Python code, you have two basic options for handling control-C.

Option 1: KeyboardInterrupt

The first option is to ignore the issue entirely. By default, the Python interpreter sets things up so that control-C will cause a `KeyboardInterrupt` exception to materialize at some point in your code, which then propagates out like any other regular exception. This is pretty nice! If your code was accidentally caught in an infinite loop, then it breaks out of that. If you have cleanup code in `finally` blocks, it gets run. It shows a traceback so you can find that infinite loop. That's the advantage of the `KeyboardInterrupt` approach: even if you didn't think about control-C at all while you were writing the program, then it still does something that's pretty darn reasonable – say, 99% of the time.

The problem is that the other 1% of the time, things break in weird ways. It's extremely difficult to write code that can correctly handle a `KeyboardInterrupt` *anywhere* and still guarantee correctness. No-one audits or tests their code for this. And the edge cases are very tricky. For example, suppose you have some code that takes and then releases a lock:

```
lock.acquire()
try:
    do_stuff()             # <-
    do_something_else()    # <- control-C anywhere here is safe
    and_some_more()        # <-
finally:
    lock.release()
```

If the user hits control-C anywhere inside the `try` block, then the resulting `KeyboardInterrupt` will cause the `finally` block to run, the lock will be released, and all will be well. But what if we're unlucky?

```
lock.acquire()
                           # <- control-C could happen here
try:
    ...
finally:
                           # <- or here
    lock.release()
```

If a `KeyboardInterrupt` happens at one of the two points marked above, then sucks to be us: the exception will propagate but our lock will *never* be released, which means that instead of exiting cleanly we might well get stuck in a deadlock. By moving the `acquire` inside the `try` block we could convert the first point into a `RuntimeError` ("attempt to release an unlocked lock") instead of a deadlock, but this isn't entirely satisfying, and doesn't help with the second point. And there's another possibility: `KeyboardInterrupt` could be raised *inside* `lock.acquire` or `lock.release` – meaning that we could end up with a lock that was "half-acquired". I'm not sure what that means but it's probably bad.

In any case, the point here is to illustrate a more general principle: most Python code has dozens of these kinds of dangerous moments when a `KeyboardInterrupt` will violate invariants. Our running example uses a lock because trio is a concurrency library, but the same thing applies to open files, database transactions, any kind of multi-step operation that mutates external state... usually you're lucky enough to get away with it, especially since the program usually exits afterwards anyway, but it's basically impossible to know for certain, so if you need 100% reliability then you need a different approach. [1]

Option 2: a custom signal handler

The problem with `KeyboardInterrupt` is that it can happen *anywhere*. If we want to make this manageable, then we need to somehow trim down the number of places where we need to think about control-C. The general strategy here is to register a custom handler for SIGINT that does nothing except set some kind of flag to record that the signal happened. This way we can be pretty confident that the signal handler itself won't interfere with whatever the program was doing when the signal handler ran. And then we have to make sure that our program checks this flag on a regular basis at places where we know how to safely clean up and exit. The best way to think about this is that we set up a "chain of custody" where responsibility for handling the signal gets handed along from tricky low-level code up to higher-level code whose execution context is better-defined:

```
custom signal handler -> our program's main loop
sets flag               checks flag
```

It's hard to say more than this, though, because the implementation is going to depend a lot on the way each particular program is put together. That's the downside to this approach: making it work at all requires insight into our program's structure and careful attention to detail. If we mess up and don't check the flag for a few seconds (perhaps because we're busy doing something else, or the program is sleeping while waiting for I/O to arrive, or ...), then oops, it takes a few seconds to respond to control-C. To avoid this we may need to invent some kind of mechanism to not just set the flag, but also prod the main loop into checking it in a timely fashion:

```
custom signal handler          -> our program's main loop
sets flag                         gets woken up by being poked with stick
& pokes main loop with a stick    & checks flag
```

Another possibility is that we *really* mess up and accidentally get stuck in an infinite loop that doesn't check for the flag, and then oops, now control-C just doesn't work at all, which is a really adding insult to injury – we've got a buggy program that's locked and chewing up our CPU, and now we can't even kill it? This is *exactly* the situation that control-C is supposed to handle! Argh! Super annoying.

Bottom line: this is the only viable way to handle interrupts 100% correctly, but getting there requires a lot of work, and if you mess up then you'll actually make things worse. For many programs it's not worth it – we may be better off letting Python do its default thing of raising `KeyboardInterrupt` and crossing our fingers.

The nice thing about Python's approach is that it gives us both options, and lets us pick the trade-offs that work best for each situation.

The dream

So those are your options in regular Python; what if you're using Trio?

In general, Trio tries to make async programming feel similar to regular Python programming, with some minimal extensions added. For example, if we want to call A and then call B we don't write some complicated thing like `fut = A(); fut.add_callback(B)`, we just write `A(); B()` (maybe with some `awaits` thrown in). Our model for running concurrent tasks is that spawning a task is similar to calling a function, except that you now you can call several functions *at the same time*. And – important for our current discussion – this means we can report errors using ordinary exceptions and the usual stack unwinding logic, even when those errors have to cross between different concurrent tasks.

For example, a simple web server might have a task tree that looks like:

```
parent task supervising the other tasks
│
├─ task listening for new connections on port 80
│
├─ task talking to client 1
│
├─ task talking to client 2
│
├─ task talking to client 3
┆
```

Now suppose we haven't defined any special control-C handling, the user hits control-C, and the second client task receives a `KeyboardInterrupt`. Then this exception will propagate up the stack inside the "client 2" task – running any cleanup code as it goes. Generally in this kind of server you'd have some sort of catch-all block near the top of the task that catches, logs, and discards most exceptions, because we don't want a typo in some HTML template to take down the whole server. But if our server is well written, this catch-all handler will only catch

`Exception` and not `BaseException` – this is just a standard Python thing, nothing to do with trio – so it won't catch the `KeyboardInterrupt` exception, which will eventually hit the top of that task's stack.

At this point, it continues to propagate up the task tree, into the supervisor task. When a supervisor sees a child crashing with an exception like this, the default response is to cancel all the other tasks and then re-raise the exception. So now all the other tasks will recieve `trio.Cancelled` exceptions, clean themselves up, and then the whole thing exits with `KeyboardInterrupt`. Nice! That's just what we wanted, and we didn't have to think about control-C handling at all when we were writing the code – it just worked.

So what this suggests is that trio should provide exactly the same semantics as regular Python: by default control-C triggers a `KeyboardInterrupt` in your code and then trio's normal exception propagation logic will take care of things, or else you can define a custom handler with some custom cleanup logic if you want to be really careful.

Now all we need to do is implement it... but this turns out to be non-trivial, because trio is itself implemented in Python. In our little scenario above, we imagined that `KeyboardInterrupt` was raised inside the user's code. But if we're unlucky, we might get a `KeyboardInterrupt` inside trio itself. For example, in trio's core scheduling loop there's a bit of code that picks the next task to run by doing something like:

```
next_task = run_queue.pop()
```

Imagine a `KeyboardInterrupt` arriving after the call to `pop()` but before the assignment! Even if we catch the error, we just lost track of this task. That's no good.

This is a bit of a theme in trio: a genuinely wonderful thing about Python's async/await design is that it's not bound to any particular event loop or execution model: it's basically just a minimal stack-switching primitive that lets us build our own cooperative threading semantics on top as an ordinary Python library. If Python's async/await looked like C#'s async/await or Javascript's async/await, then libraries like trio and curio couldn't exist, because asyncio would be baked into the language. But... it turns out that trying to extend the Python runtime's core semantics, in Python, is a great way to discover all kinds of interesting edge cases!

Can we do better?

Prior art

Do other async libraries give any useful hints on what to do? Not really, unfortunately.

Twisted

Twisted by default registers a signal handler for control-C that triggers a clean shutdown of their event loop. This means that control-C won't work if your Twisted program runs away in an infinite loop that never yields to the event loop, and even if does work then any callback chains or coroutines that are in progress will get abruptly abandoned, but it will at least run any registered shutdown callbacks. It's not bad, it can be made to work, but doing so is tricky and there are limitations. Trio's motto is "make it easy to get things right", so we'd like to do better.

Other async libraries

I also looked at tornado, asyncio, curio, and gevent, but (as of April 2017) they're even less sophisticated than twisted: by default they don't do any special handling for keyboard interrupts at all, so hitting control-C may or may not blow up their event loop internals in a graceless fashion; in particular, any callback chains or coroutines you have running are likely to be abruptly abandoned, with no chance to even run their `finally` blocks, and it's entirely possible that you'll hit a deadlock or something, who knows. And as an additional wrinkle, at least asyncio has some problems handling control-C on Windows. (Checked with asyncio in CPython 3.6.1; I didn't

check the other projects at all.) For example, if you run this program then be prepared to kill it with the task manager or something, because your control-C has no power here:

```
# On Windows this ignores control-C, so be prepared to kill it somehow...
import asyncio
loop = asyncio.get_event_loop()
loop.run_until_complete(asyncio.sleep(99999))
```

You can implement the Twisted-style behavior on these systems by manually registering your own signal handler that triggers some graceful shutdown logic, but all in all it's not very user friendly, and has the same limitations. (The asyncio developers have even considered making the Twisted-style behavior the default, but are unhappy about the side-effects and haven't reached consensus on a solution.)

How does the Python interpreter pull it off?

We do have one example of a program that implements the semantics we want: the Python interpreter itself. How does it work? Let's walk through it.

Control-C handling starts when the operating system detects a control-C and informs the interpreter. The way it does this is by running whatever signal handler was previously registered to handle the `SIGINT` signal. Conceptually, this is similar to how signal.signal works, but technically it's very different because `signal.signal` takes a *Python* function to be run when a signal arrives, and the operating system APIs only let you register a *C* function to be run when a signal arrives. (Note that here we're talking about "C" the language – that it uses the same letter as control-C is just a coincidence.) So if you're implementing a Python interpreter, that's your challenge: write a function in C that causes the Python signal handler function to be run. Once you've done that, you're basically done; to get Python's default behavior you just have to install a default handler that looks like:

```
def default_sigint_handler(*args):
    raise KeyboardInterrupt
```

and then if the user wants to override that with something fancier, they can.

But implementing the C-level handler turns out to be trickier than you might think, for the same basic reason we keep running into: control-C can happen at *any* moment. On Unix, signal delivery is done by hijacking a thread, essentially pausing it in between two assembly instructions and inserting a call to a C function that was registered as a signal handler. (What if the thread isn't running any assembly instructions, because it's blocked in a syscall inside the kernel? Then the kernel unceremoniously cancels that syscall – making it return the special error code `EINTR` – and this forces the thread back into userspace so it can be hijacked. Remember that stick we mentioned above? The kernel has a very big stick. This design is historically somewhat controversial [2].) On Windows, things are a bit more civilized and also more annoying: when the user hits control-C, a new thread spontaneously materializes inside our process and runs the C signal handler. On the one hand, this is an elegant re-use of an existing concept and avoids the whole weird hijacking thing. On the other hand, if you want to somehow poke the main thread to wake it up, then you're on your own – you have to build your own stick from scratch.

In any case, the end result of all this is that the C-level signal handler will get run, *but* this might happen a time when the interpreter is in some messy and inconsistent state. And in particular, this means that you can't simply have the C-level signal handler run the Python-level signal handler, because the interpreter might not be in a state where it can safely run Python code.

To see why this is a problem, let's look at an example from inside CPython. When raising an exception, Python keeps track of three things: the exception's type, value, and traceback. Here's the code from `PyErr_SetExcInfo` that CPython uses to record these (comments are mine; original is here):

```
/* Save the old exc_info values in temporary variables */
oldtype = tstate->exc_type;
```

```
oldvalue = tstate->exc_value;
oldtraceback = tstate->exc_traceback;

/* Assign the new exc_info values */
tstate->exc_type = p_type;
tstate->exc_value = p_value;
tstate->exc_traceback = p_traceback;

/* Drop the references to the old values */
Py_XDECREF(oldtype);
Py_XDECREF(oldvalue);
Py_XDECREF(oldtraceback);
```

You'll notice this is written in a slightly complicated way, where instead of simply overwriting the old values, they get saved in temporaries etc. There are two reasons for this. First, we can't just overwrite the old values because we need to decrement their reference counts, or else we'll cause a memory leak. But we can't decrement them one by one as we assign each field, because `Py_XDECREF` can potentially end up causing an object to be deallocated, at which point its `__del__` method might run, which is arbitrary Python code, and as you can imagine you don't want to start running Python code at a moment when an exception is only *half* raised. Before it's raised is okay, after it's raised is okay, but half-way raised, with `sys.exc_info()` only partially filled in? That's not going to end well. The CPython developers of course are aware of this, so they carefully wrote this function so that it assigns all of the values and puts the interpreter back into a sensible state before it decrements any of the reference counts.

But now imagine that a user is annoying (as users sometimes are) and hits control-C right in the middle of this, so that just as we're half-way through assigning the new values, the operating system pauses our code and runs the C signal handler. What happens? If the C-level signal handler runs the Python-level signal handler directly, then we have the same problem that we just so carefully avoided: we're running arbitrary Python code with an exception only half-raised. Even worse, this Python function probably wants to raise `KeyboardInterrupt`, which means that we end up calling `PyErr_SetExcInfo` to raise a second exception while we're half-way through raising the first. Effectively the code would end up looking something like:

```
/*****************************************************************/
/* Raising the first exception, like a RuntimeError or whatever   */

/* Save the old exc_info values in temporary variables */
oldtype1 = tstate->exc_type;
oldvalue1 = tstate->exc_value;
oldtraceback1 = tstate->exc_traceback;

/* Assign the new exc_info values */
tstate->exc_type = p_type1;

    /*****************************************************************/
    /* Surprise! Signal handler suddenly runs here, and calls this    */
    /* code again to raise a KeyboardInterrupt or something           */

    /* Save the old exc_info values in temporary variables */
    oldtype2 = tstate->exc_type;
    oldvalue2 = tstate->exc_value;
    oldtraceback2 = tstate->exc_traceback;

    /* Assign the new exc_info values */
    tstate->exc_type = p_type2;
    tstate->exc_value = p_value2;
    tstate->exc_traceback = p_traceback2;

    /* Drop the references to the old values */
    Py_XDECREF(oldtype2);
    Py_XDECREF(oldvalue2);
    Py_XDECREF(oldtraceback2);
```

```
/*****************************************************************/
/* Back to the original call                                     */

tstate->exc_value = p_value1;
tstate->exc_traceback = p_traceback1;

/* Drop the references to the old values */
Py_XDECREF(oldtype1);
Py_XDECREF(oldvalue1);
Py_XDECREF(oldtraceback1);
```

This would cause all kinds of chaos: notice that `p_type2` overwrites `p_type1`, but `p_value1` overwrites `p_value2`, so we might end up with a `sys.exc_info()` where the type is `KeyboardInterrupt` but the exception object is an instance of `RuntimeError`. The `oldvalue1` and `oldvalue2` temporaries end up referring to the same object, so we end up decrementing its reference count *twice*, even though we only had one reference; this probably leads to some kind of nasty memory corruption.

Clearly this isn't gonna work. The C-level signal handler cannot call the Python-level signal handler directly. Instead, it needs to use the same trick we discussed above: the C-level handler sets a flag, and the interpreter makes sure to check this flag regularly at moments when it knows that it can safely run arbitrary Python code.

Specifically, the way CPython does this is that in its core bytecode evaluation loop, just before executing each bytecode instruction, it checks to see if the C-level handler's flag was set, and if so then it pauses and invokes the appropriate Python handler. (After all, the moment when you're about to run an arbitrary opcode is by definition a moment when you can run some arbitrary Python code.) And then, if the Python-level handler raises an exception, the evaluation loop lets this exception propagate instead of running the next instruction. So a more complete picture of our chain of custody looks like this, with two branches depending on which kind of Python-level handler is currently set. (These correspond to the two strategies we described at the beginning.):

```
C-level handler --> bytecode eval loop
sets flag           checks flag & runs Python-level handler
                                       | \
                                       |  default Python-level handler
                                       |  raises KeyboardInterrupt
                                        \
                                         \
                                          custom Python-level handler --> main loop
                                          sets another flag               checks flag
```

But what if the eval loop isn't actually... looping? What if it's sitting inside a call to `time.sleep` or `select.select` or something? On Unix this is mostly taken care of automatically by the kernel – though at the cost of the interpreter needing annoying boilerplate every time it does an operating system call. On Windows, we're on our own. And unfortunately, there is no general solution, because, well, it's Windows, and the Windows low-level APIs wouldn't recognize "general" if it showed up in a uniform with stars on the shoulder. Windows has at least 4 qualitatively different methods for interrupting a blocking call, and any given API might respond to one, several, or none of them [3].

In practice CPython compromises and uses two mechanisms: the C-level handler can be configured to write to a file descriptor (which is useful for waking up calls that wait for a file descriptor to have data, like select), and on Windows it unconditionally fires an "event" object, which is a Windows-specific synchronization primitive. And some parts of CPython are written to check for this – for example the Windows implementation of `time.sleep` is written to wake up early if the event gets fired and check for signals. And that's why on Windows you can do `time.sleep(99999)` and then hit control-C to cancel it. But this is a bit hit-and-miss: for example, Python's implementation of `select.select` doesn't have any similar early-exit code, so if you run this code on Windows and hit control-C, then it will raise `KeyboardInterrupt`... a month from now, give or take:

```
# If you run this on Windows, have the task manager ready
sock = socket.socket()
select.select([sock], [], [], 2500000)
```

The C-level signal handler runs and sets its flag, but the interpreter doesn't notice until the `select` call has finished. This explains why asyncio has problems – it blocks in `select.select`, not `time.sleep`. Which, I mean, that's what you *want* in an event loop, I'm not saying it should block in `time.sleep` instead, but if you're using `select.select` then Python's normal guarantees break down and asyncio isn't compensating for that.

So here's the *final* version of our chain-of-custody diagram for control-C in a generic Python program:

```
C-level handler --> bytecode eval loop
sets flag           checks flag & runs Python-level handler
& writes to fd                         | \
  (if enabled)                         |  default Python-level handler
& fires an event                       |  raises KeyboardInterrupt
  (if on windows)                       \
                                         \
                                          custom Python-level handler --> main loop
                                          sets another flag               checks flag
```

And now you know how the Python runtime handles control-C (usually) promptly and reliably, while protecting itself from getting into a broken state.

Of course, this doesn't really help the code that's running on top – if your Python code wants to avoid getting wedged in a broken state, it's on its own.

...Mostly. It turns out that that there are some details that can sometimes make our Python code a little more robust to `KeyboardInterrupts`. There's no guarantee – remember, this is the 99% solution we're trying to implement – but if the interpreter can make it 99.9% instead of 99.0% without any extra work for users, then it's a nice thing to do (and we probably want to do the same thing in trio, if we can). So let's look at how these work.

Let's start with our example from above, of some code that isn't quite `KeyboardInterrupt` safe:

```
lock.acquire()
try:
    ...
finally:
    lock.release()
```

First, what happens if `KeyboardInterrupt` is raised when we're half-way through running `lock.acquire` or `lock.release`? Can we end up with our lock object in an inconsistent state where it's only "half-locked" (whatever that would even mean)?

Well, *if* our lock is an instance of the standard library's `threading.Lock` class, then it turns out we're safe! `threading.Lock` is implemented in C code, so its methods get the same kind of protection that `PyErr_SetExcInfo` does: you can get a `KeyboardInterrupt` before or after the call, but not during the call [4]. Sweet.

What about a `KeyboardInterrupt` that happens between calling `acquire` and entering the `try` block, or between entering the `finally` block and calling `release`? Well, in current CPython there's no way to eliminate this entirely, but it turns out that the bytecode eval loop has some tricks up its sleeve to make things less risky.

The first trick we'll examine is also the oldest, and probably the least useful. To see how this works, we need to look at how our example gets compiled down to bytecode instructions that run on CPython's virtual machine. (If you aren't familiar with CPython's bytecode, this is a great talk and will give you a good introduction.) Running this code:

```
import dis
def f():
```

```
    lock.acquire()
    try:
        pass
    finally:
        lock.release()
dis.dis(f)
```

prints a chunk of disassembled bytecode. I won't paste the whole thing, but it starts like:

```
2           0 LOAD_GLOBAL              0 (lock)
            3 LOAD_ATTR                1 (acquire)
            6 CALL_FUNCTION            0 (0 positional, 0 keyword pair)
            9 POP_TOP

3          10 SETUP_FINALLY            4 (to 17)
```

The first four lines of bytecode correspond to the first line of our Python code, the call to `lock.acquire()`. Then SETUP_FINALLY marks the beginning of the `try` block. So danger here would be if a `KeyboardInterrupt` arrives in between the CALL_FUNCTION (where we actually acquire the lock) and the SETUP_FINALLY. Since signal handlers run in between opcodes, there are two places this could happen: between CALL_FUNCTION and POP_TOP, and between POP_TOP and SETUP_FINALLY.

Well, it turns out that way back in 2003, Guido added a bit of code to the bytecode eval loop to skip running signal handlers if the next opcode is SETUP_FINALLY, and it's still there today. This means that we can't get a `KeyboardInterrupt` in between POP_TOP and SETUP_FINALLY. It's... mostly useless? We can still get a `KeyboardInterrupt` in between CALL_FUNCTION and POP_TOP, and in fact the CALL_FUNCTION → POP_TOP case is much more likely to cause problems then that POP_TOP → SETUP_FINALLY case. The check after CALL_FUNCTION notices any signals that arrived during CALL_FUNCTION, which can take an arbitrarily long time; the check after POP_TOP only notices signals that arrived during POP_TOP, and POP_TOP is an extremely fast opcode – basically just a few machine instructions. In fact it's so fast that the interpreter usually doesn't bother to check for signals after it *anyway* because the check would add substantial overhead [5], so in our example this special case doesn't really accomplish anything at all.

The one case I can think of where the SETUP_FINALLY special case might be useful is in code like:

```
SOME_VAR = True
try:
    ...
finally:
    SOME_VAR = False
```

because if you look at how this compiles to bytecode, the assignment ends up being a single opcode that comes right before the SETUP_FINALLY. But fundamentally, this strategy can't really work: there's generally going to be *some* sort of logically atomic operation before each `try`/`finally` pair that shouldn't be interrupted by signals, but there's no way for the interpreter to figure out where the start of that of that logical operation is. That information just isn't recorded in the source code.

Except... sometimes it is, which leads to another trick the interpreter pulls. Back in 2003 `try`/`finally` was all we had, but in modern Python, a nicer way to write our example would be:

```
with lock:
    ...
```

Of course it's well documented that this is just syntactic sugar for something like:

```
# simplified but gives the idea, see PEP 343 for the full details
lock.__enter__()
try:
    ...
```

```
finally:
    lock.__exit__(...)
```

This looks pretty similar to our problematic code above, so one would think that the `with` version has the same problems. But it turns out this is not quite true – not only is the `with` version nicer to look at it than the `try`/`finally` version, it actually makes stronger guarantees about `KeyboardInterrupt` safety!

Again, let's look at the bytecode:

```
import dis
def f():
    with lock:
        pass
dis.dis(f)
```

```
2           0 LOAD_GLOBAL              0 (lock)
            3 SETUP_WITH               5 (to 11)
            6 POP_TOP

3           7 POP_BLOCK
            8 LOAD_CONST               0 (None)
      >>   11 WITH_CLEANUP_START
           12 WITH_CLEANUP_FINISH
           13 END_FINALLY
```

The key thing we learn here is that entering a `with` block is done via SETUP_WITH and exiting is done via WITH_CLEANUP_START. If we consult `Python/ceval.c` in the CPython source, it turns out that SETUP_WITH is a single opcode that both calls `lock.__enter__` and also sets up the invisible `try` block, and WITH_CLEANUP_START is a single opcode that both marks the beginning of the invisible `finally` block and also calls `lock.__exit__`. And the crucial thing for us is that since the interpreter only runs Python-level signal handlers *in between* opcodes, this means it's now impossible for a `KeyboardInterrupt` to arrive in between calling `lock.__enter__` and entering the `try` block, or in between entering the `finally` block and calling `lock.__exit__`.

Basically, the key thing about `with` blocks is that they tell the interpreter where the boundary of the critical operations are (they're whatever `__enter__` and `__exit__` do) so a solution becomes possible in principle; then `threading.Lock.__enter__` is implemented in C so it's atomic itself, and the design of the `with` opcodes rules out the two remaining problematic cases: `KeyboardInterrupt` after acquiring the lock but entering the `try`, and `KeyboardInterrupt` after entering the `finally` but before releasing the lock. Hooray, we're safe!

...almost. Now we can't have a `KeyboardInterrupt` between entering the `finally` block and releasing the lock. But that's not really what we want. We want to make sure we can't have a `KeyboardInterrupt` between exiting the `try` block and releasing the lock. But wait, you might think. This is really splitting hairs – just look at the source code, the end of the `try` block and the start of the `finally` block are the same thing!

Well, yeah, that would make sense... but if we look at the bytecode, we can see that this isn't quite true: the POP_BLOCK instruction at offset 7 is the end of the `try` block, and then we do a LOAD_CONST before we reach the WITH_CLEANUP_START at offset 11, which is where the `finally` block starts.

The reason the bytecode is written like this is that when the interpreter gets to the `finally` block hidden inside WITH_CLEANUP_START, it needs to know whether it arrived there because an exception was thrown or because the `try` block finished normally. The LOAD_CONST leaves a special value on the stack that tells WITH_CLEANUP_START that we're in the latter case. But for present purposes the reason doesn't really matter... the end result is that there's this gap, where if we get a `KeyboardInterrupt` raised in between the POP_BLOCK and LOAD_CONST, or in between the LOAD_CONST and WITH_CLEANUP_START, then it will propagate out of the `with` block without calling `__exit__` at all. Oops!

Bottom line: even if you use a `with` block AND use a lock that's implemented in C, it's still possible for a control-C to happen at *just* the wrong moment and leave you with a dangling lock. And of course, there are many

other ways that a poorly timed `KeyboardInterrupt` can trip you up; even if this particular case were fixed (which would be nice!), then this doesn't provide a general solution to those problems. But if we accept that the default `KeyboardInterrupt` handling is a best-effort kind of thing, then this kind of extra safety is still a nice bonus when we can get it – and in particular using `with` and a lock implemented in C is much less likely to break than using `try/finally` with a lock implemented in Python, so we should appreciate the CPython developers for taking the effort.

How Trio handles control-C

Ok, that's enough about regular Python – this is a blog post about trio! And as we discussed above, trio has the same basic problem that CPython does: we want to provide `KeyboardInterrupt` semantics to code running on top of us, but our internal code that implements low-level runtime services like scheduling, exception propagation, and locking, is too delicate to survive random `KeyboardInterrupts` without breaking the whole system. If trio were built into the interpreter, then this would be no problem, because like we saw above, the interpreter can (and must!) cheat to make operations like `PyErr_SetExcInfo` and `threading.Lock.__enter__` atomic with respect to signal delivery. But trio is an ordinary library written in pure Python, so we don't have this option. What to do?

Okay, enough buildup. Here's how trio handles control-C:

First, we jump through some hoops to make sure that the hand-off from the C-level signal handler to the Python-level signal handler happens promptly, even on Windows. Basically this just means that whenever we stop executing Python bytecode because we're waiting for I/O, we make sure to hook up to one of the wakeup signals that C-level signal handler sends. You can read the gory details on the trio bug tracker. This is just a baseline requirement to get any kind of reliable signal handling in Python.

Next, trio checks at startup to see if the user has configured their own custom SIGINT handler. If not, then we figure they're expecting the Python style semantics, and we automatically replace the default handler with trio's custom handler.

Conceptually, trio's handler is similar to the interpreter's default handler: its goal is to respond to control-C by raising `KeyboardInterrupt` inside user code, but not inside the delicate parts of the runtime – just now it's trio's runtime we're worried about protecting, not the underlying language runtime. But unfortunately, we can't copy CPython's trick of waiting until user code is running before calling the signal handler – that's just not functionality that Python offers. Python will call our signal handler whenever it wants, and we can't stop it. So when our signal handler gets called, its first job is to figure out whether it's user code that got interrupted. If so, then it raises `KeyboardInterrupt` directly – which lets us break out of that accidental infinite loop I keep talking about – and we're done done. Otherwise, it sets a flag and wakes up the run loop to deliver a `KeyboardInterrupt` as soon as possible.

So two questions: how does it know whether it's being called from "user code"? and if we can't deliver a `KeyboardInterrupt` immediately, then how do we deliver it "as soon as possible"?

How do we know which code should be protected?

The most important code we need to protect from `KeyboardInterrupt` is the core scheduling code that runs to switch between user tasks. So my first thought was that we could have a global flag the keeps track of whether protection is "enabled" or "disabled", and toggle it back and forth when scheduling a user task. Something like:

```
KEYBOARD_INTERRUPT_PROTECTION_ENABLED = True

def run_next_task_step(task):
    # Disable protection just while we're running the user task,
    # and re-enable immediately afterwards:
    global KEYBOARD_INTERRUPT_PROTECTION_ENABLED
```

```
KEYBOARD_INTERRUPT_PROTECTION_ENABLED = False
try:
                                                    # <- danger zone!
    # Run this task for one step:
    return task.coro.send(task.next_value_to_send)
finally:
                                                    # <- danger zone!
    KEYBOARD_INTERRUPT_PROTECTION_ENABLED = True
```

But if you've read this far, then this `try`/`finally` block should look pretty familiar, and you can probably guess the problem! What if our signal handler runs at one of the places labeled "danger zone"? In both places the protection is disabled, so a `KeyboardInterrupt` can be raised... but if it is, then we're in trouble. If we call `run_next_task_step` and get back a `KeyboardInterrupt`, then that might mean that we didn't run the task at all, or it might mean that the task itself raised `KeyboardInterrupt`, or it might mean that we did run the task step and then lost the return value... and we can't tell the difference or recover the return value. So this doesn't work at all! What we need is some way to combine task switching and the enabling/disabling of `KeyboardInterrupt` protection into a *single atomic operation*, without cheating and using C code.

This stumped me for a while, but it turns out that this is actually possible. Here's the trick: Python signal handlers receive an obscure second argument, which is the stack frame of the function whose execution was paused in order to run the signal handler. This frame is either inside the user task, or inside trio's scheduler. If our signal handler can somehow examine this frame and figure out which type it is, then the handler will know whether it's safe to raise `KeyboardInterrupt`. And crucially, by tying this decision to the frame object, we make it so that the actual act of switching in or out of the user task is what toggles the protection, so there's no moment where our protection is disabled inside the scheduler.

So now our problem becomes: how do we "mark" a stack frame as protected or unprotected? My first thought was to stick a special attribute on functions that transition between the two modes, and then the signal handler could walk up the stack looking for this special attribute. But unfortunately, it turns out that there's no way to get from a stack frame object back to a function object to look at its attributes. And there's no way to attach generic metadata to frame objects. (They don't have a `__dict__`, and while code objects do have a `flags` attribute, it's read-only from Python. Of course nothing is ever REALLY read-only in Python, but stealing one of CPython's code flags to use in a third-party library might be considered rude...) In fact, it turns out that there's only one place where we can attach arbitrary user-defined data to a frame object, and that's in the local variables!

(In case you ever wondered why pytest uses a magic variable `__tracebackhide__` as its mechanism to mark functions that shouldn't show up in tracebacks, this is why. This is also why tracebacks don't show class names on methods – that information is stored in the method object's `__qualname__` attribute, but there's no reasonable way to get from a traceback back to the method object.)

Anyway, since it's our only option, that's what trio does: we define a special sentinel value as the "name" of our local variable. (It's not a string, to make sure we don't accidentally clash with real user variables – it turns out Python is fine with this, because the locals namespace is just a dict, and like all dicts it accepts any random hashable object as a key.) Whenever we start a user task, we stash a setting for this variable into the task's top stack frame. Then when our signal handler runs, it can walk up the stack and when it sees the magic variable, that tells it whether or not to raise `KeyboardInterrupt`. The details here aren't public APIs and are subject to change, but that's how it works.

Then to handle cases like `trio.Lock.__enter__` we also have a decorator that can be used to mark a function as needing protection against `KeyboardInterrupt`. (And under the hood, of course, it also works by setting up a magic local variable where our stack introspection logic can find it.) It's not recommended for use in end-user code, because if you care enough about control-C to take these kinds of special measures, then you're almost certainly better off with a generic solution (see below) than playing whack-a-mole with individual functions. But internally trio uses this on all of its functions that manipulate inter-task state to minimize the chances that an untimely control-C will wedge the whole system, and it's a public API in case you want to implement your own synchronization primitives.

How do we deliver a KeyboardInterrupt if we can't raise it?

So now we know how trio's signal handler decides whether it's OK to throw a `KeyboardInterrupt` directly into the code that's currently running. But what if it decides that it's not safe? What do we do then? Really the only thing we can do is to set some sort of flag and arrange for it to be delivered later.

Fortunately, trio has a generic cancellation system that's designed to do things like raise an exception if some code exceeds its timeout. So we've already solved the problem of finding a good place to deliver the exception (we call these "checkpoints"), implemented a mechanism for waking up a sleeping task if necessary to make the delivery, and provided well-defined semantics for these exceptions. All trio code already has to be prepared to handle `Cancelled` exceptions at checkpoints, and `Cancelled` and `KeyboardInterrupt` are very similar. (They even both inherit from `BaseException` instead of `Exception`, because in both cases the only legal way to handle them is to clean up and let them propagate.)

So if a `KeyboardInterrupt` happens at a moment when we can't deliver it directly, we instead hand it off to the cancellation system to deliver for us:

```
C-level handler -> bytecode eval loop
sets flag          checks flag
& wakes loop       & runs Python-level handler
                          \
                           trio's Python-level handler
                           raises KeyboardInterrupt
                                  -- or --
                           sets flag and wakes task ---> trio's cancellation machinery
                                                         raises KeyboardInterrupt
                                                         at next checkpoint
```

This picture is still somewhat simplified, and omits several of the trickier variations. For example, during startup and shutdown there are brief periods where trio can receive a control-C signal but there aren't any tasks running to deliver it to. [6] But that's all solved, and we have an exhaustive set of tests to make sure that the handoff chain is never broken and that no `KeyboardInterrupt` is accidentally lost.

What if you want a manual control-C handler?

Let's pause a moment and recap. We started out discussing the two basic strategies Python programs can use to handle control-C: the easy and mostly effective default of getting a `KeyboardInterrupt` raised at some arbitrary location, and the more difficult and fragile but also potentially safer option of installing a custom handler and then implementing some sort of hand-off chain to make sure that it promptly triggers some kind of clean shutdown logic. Now you've heard how CPython implements these two strategies internally, and how trio implements the first strategy. That's good enough for most trio users – ignore the problem and everything will mostly work out :-). But what about if you're using trio, and you're paranoid enough that you want the second strategy? How can trio help you?

It turns out that implementing this kind of safe control-C handling is actually much easier for trio programs than for generic Python programs, because you get a lot of the necessary infrastructure for free. Trio's API for catching signals gives you a simple and reliable way to get signal notifications *in a regular task context*, letting you skip all the tricky bits required when writing your own custom signal handler. Then after you're notified, you can respond however you want – but if what you want is to just shut everything down in a clean fashion, then again, trio's infrastructure can do most of the work for you. In a typical trio program it might look like:

```
# This works, but we'll discuss an even easier way below:

async def main():
    async with trio.open_nursery() as nursery:
        if threading.current_thread() is threading.main_thread():
            # Spawn a child to watch for control-C
```

```
            nursery.spawn(control_c_watcher)
        # ... spawn other children to do the real work ...

async def control_c_watcher():
    async with trio.catch_signals({signal.SIGINT}) as sigset_aiter:
        async for _ in sigset_aiter:
            # the user hit control-C
            raise KeyboardInterrupt

trio.run(main)
```

The idea here is that we spawn a child task that does nothing but sit and wait for a control-C to happen, at which point it raises `KeyboardInterrupt`. So if this is our example HTTP server from above, we'd end up with a task tree like:

```
task supervising the other tasks
│
├─ task waiting for control-C                    # <-- this is new
│
├─ task listening for new connections on port 80
│
├─ task talking to client 1
│
⋮
```

And as we discussed, trio implements sensible default behavior for exception propagation, so if the `control_c_watcher` task raises `KeyboardInterrupt`, then the `main` task supervisor will notice this and cleanly cancel the other tasks. This is the same behavior that makes the default `KeyboardInterrupt` handling useful; the difference here is that now the *only* place that `KeyboardInterrupt` can be raised is inside `control_c_watcher`, so we don't have to worry about it interrupting some delicate state manipulation inside our real logic.

That said, this is still a bit of extra work to set up, and has some potential pitfalls – for example, `catch_signals` can only be used if we're running inside the main Python thread (this is a general restriction on Python's signal handling functions), so we had to remember to check that before spawning the watcher task. This is already so, so, so much easier than the equivalent in most other frameworks, and it has the advantage that it allows total flexibility in how you respond to the signal – for example, you could use pretty much the same code to watch for SIGHUP and then reload the server's configuration, instead of shutting down. But in the normal control-C case where we want to raise `KeyboardInterrupt` and shut everything down... can trio help you *even more*?

Well, while I was writing this section I realized that yeah, actually, it could :-). Remember how earlier we learned that when trio's custom signal handler can't deliver a `KeyboardInterrupt` directly, then as a fallback it routes it through trio's cancellation system? That system that's carefully designed to allow arbitrary code execution to be cancelled in a safe and controlled way? What if we, just... always did that?

Every trio program starts with a line like:

```
trio.run(main)
```

Starting in the next release (v0.2.0), you can instead write:

```
trio.run(main, restrict_keyboard_interrupt_to_checkpoints=True)
```

and this toggles the behavior of trio's control-C handler so that it *always* routes `KeyboardInterrupt` through the cancellation system. Basically this is just taking the protection that trio uses for its own internals, and extending it over your whole program; the implementation is one line long.

The end result is that if you turn this on, then your program only needs to handle `KeyboardInterrupt` at certain well-defined places called checkpoints – and these are exactly the same places where your program needs to be

prepared to receive a `Cancelled` exception anyway, e.g. because a timeout expired, so the extra work is essentially zero. It's still not enabled by default, because if you turn it on then runaway loops like

```
while True:
    pass
```

can't be interrupted (there's no checkpoint inside the loop), and because it's not what users expect when coming from Python. But if you want the safety of a custom signal handler, this lets you have the safety without the complexity. Pretty sweet.

Limitations and potential improvements

Unfortunately, even trio's control-C handling is not (yet) perfect – mostly due to bugs and limitations in the Python interpreter. Here are some notes on the issues I've run into so far. For reference, here's the handoff chain diagram again – I find it useful to look at while thinking about these things, because the bugs here all involve something going wrong along that path:

```
C-level handler -> bytecode eval loop
sets flag          checks flag
& wakes loop       & runs Python-level handler
                          \
                           trio's Python-level handler
                           raises KeyboardInterrupt
                                  -- or --
                           sets flag and wakes task ---> trio's cancellation machinery
                                                         raises KeyboardInterrupt
                                                         at next checkpoint
```

Issues with handing off from the C-level handler to the Python-level handler

bpo-30038: This is a bug in the logic used to hand-off from the C-level signal handler to the Python-level signal handler: it turns out that the C-level signal handler pokes the main thread to wake it up *before* it sets the flag to tell it that there's a signal pending. So on Windows where the C-level handler runs in its own thread, then depending on how the kernel schedules things, sometimes the main thread gets woken up, checks for signals, sees that the flag is *not* set, goes back to sleep... and then the flag gets set, but it's already too late. The main effect is that on Windows you might sometimes have to hit control-C twice before trio will notice. No workaround seems to be possible inside trio; I've submitted a fix for CPython.

bpo-30050: It turns out that Python's "wakeup fd" logic wasn't quite designed to be used in the way that trio's using it; this bug reflects a bit of behavior that made sense for the original use case, but is annoying for trio. Because of this, trio currently only uses the wakeup fd on Windows, not on Unix. This is mostly fine, because on Unix, we mostly don't need it – signals usually interrupt syscalls and wake up the main thread all by themselves. But there are some rare cases where it'd be useful even on Unix. The impact here is pretty low, and there are workarounds possible, though they have their own mildly annoying side-effects. So it's not a huge deal, but shouldn't be hard to fix either; hopefully this will get fixed for Python 3.7 so we won't have to make these compromises.

As far as I can tell, fixing these two issues should make the hand-off from the C-level handler to the Python-level handler rock-solid on all platforms.

Issues with the interaction between KeyboardInterrupt and `with` blocks

It'd be nice if code like

```
lock = trio.Lock()
```

```
async with lock:
    ...
```

could be guaranteed to release the lock even in the face of arbitrary `KeyboardInterrupts`. Unfortunately, there are currently two issues preventing this.

The first is bpo-29988: remember how in our examination of CPython's bytecode, we discovered that it's possible for a `KeyboardInterrupt` at the wrong moment to cause a `with` block to be exited without running `__exit__`? I think this is a pretty surprising violation of `with`'s semantics – and it turns out that for `async with` the race condition is actually a little worse, because its bytecode has more unprotected machinery at entry and exit to handle `awaiting` the `__aenter__` and `__aexit__` methods. This is something that can only be fixed inside the interpreter, and this is the bug to track that.

The second problem doesn't have a bug number yet, because the solution isn't as obvious. Here's the problem: remember how pleased I was to realize that by using a magic local variable to mark which stack frames are "user code" versus "internal code", we could make it so that switching stack frames also atomically toggles control-C protection on and off? That's fine for when we want to toggle protection upon switching to an *existing* stack frame, but has a problem when creating a new stack frame, like `__exit__` methods do. `trio.Lock.__exit__` effectively looks like:

```
async def __aexit__(self):
                                # <-- danger zone!
    locals()[protection_enabled_marker_object] = True

    # .. actual logic here ...
```

This is enough to make sure that the lock never gets left in an *inconsistent* state where it's only "half-locked" – either `__aexit__` runs or it doesn't, and it can't get a `KeyboardInterrupt` in the middle. But if a control-C arrives at the point marked "danger zone!" then our unlock might get cancelled before it starts. The problem is that Python doesn't really provide any decent way to attach a special value to a stack frame *at the moment its created*. Potential workarounds would be to have the signal handler introspect the current bytecode instruction pointer and treat the stack frame as protected if it looks like it's about to execute the protection code, or to have our magic decorators rewrite code objects to set the magic local as a default kwarg, since argument processing does seem to be atomic with respect to frame setup. So far I haven't attempted this because both options are pretty awkward, and at the moment it hardly seems worth the effort given that `with` and `async with` blocks always have interpreter-level race conditions.

What I'd really like to see would be for frame objects to retain a pointer to the function object that was called to create them (if any). That would:

- Fix the signal atomicity problem.
- Let me throw away the awful awful code currently required to implement the `KeyboardInterrupt` protection decorators [7] and replace it with something like:

  ```
  def enable_ki_protection(fn):
      fn._ki_protection_enabled = True
      return fn
  ```

- Bonus: pytest could potentially do something similar, instead of the odd `__tracebackhide__` thing they do now.
- Bonus: tracebacks could start including class names, so instead of:

  ```
  File "/home/njs/trio/trio/_sync.py", line 374, in acquire_nowait
    return self._lock.acquire_nowait()
  File "/home/njs/trio/trio/_sync.py", line 277, in acquire_nowait
    raise WouldBlock
  ```

we could have (notice the method names on the right):

```
File "/home/njs/trio/trio/_sync.py", line 374, in Condition.acquire_nowait
  return self._lock.acquire_nowait()
File "/home/njs/trio/trio/_sync.py", line 277, in Lock.acquire_nowait
  raise WouldBlock
```

Or maybe there's a better option, I dunno – it's just an idea. But something like this sure would be nice.

Anyway. If these two issues were fixed, then we could guarantee that `async with` was signal-safe for trio objects (and also built-in objects like `threading.Lock`, for that matter!).

yield from and await aren't signal-safe

bpo-30039: Remember up above how I said that our local variable trick works when for switching stack frames, because that's an atomic operation? Actually I lied... currently in CPython, resuming a coroutine stack is *not* atomic.

If we have coroutines calling each other, A → await B → await C, then when we do `A.send(...)`, that resumes A's frame, and then A does `B.send(...)`, which resumes B's frame, and then B does `C.send(...)`, which resumes C's frame, and then C continues executing.

The problem is that the interpreter *checks for signals in between each of these steps*, so if the user hits control-C in the middle of that sequence, then Python will raise `KeyboardInterrupt` inside A or B, and just completely forget about the rest of the call stack that it's supposed to be executing. And this affects every use of `await` or `yield from`, not just trio.

But the good news is that this is easy to fix. Remember up above how we found that CPython has a special hack where it doesn't run signal handlers if it's about to execute a SETUP_FINALLY instruction? For SETUP_FINALLY we concluded that this mostly doesn't accomplish anything, but it turns out that this is *exactly* what we need here: if we extend that check so it also skips running signal handlers before a YIELD_FROM instruction, then it fixes this bug. I've submitted this fix as a pull request.

What about PyPy?

We spent an awful lot of time above grovelling around in internal implementation details of CPython. Trio also works on PyPy: what happens there?

Answer: ¯_(ツ)_/¯ ...PyPy's, like, really complicated, ok?

Or in a little more detail: the trio testsuite passes on PyPy, and overall I've run into fewer bugs than I have on CPython (e.g. PyPy writes to the wakeup fd at the proper time, and their `throw` seems to work properly). But when it comes to the fiddly details about when exactly they check for signals, and how that's affected by JIT inlining and other transformations they apply, I currently have no idea. Maybe they'll read this blog post and help me out.

Conclusion

Now you know basically everything there is to know about signal handling in Python and Trio! You don't actually need to know any of this to use either of them, but I think it's pretty neat.

And the end result is my absolute favorite kind of feature, because it's totally invisible: it takes thousands of words to explain, but most users don't need to know about it *at all*. Trio's goal is to make it easy to get things right, and this is the ultimate example of that philosophy: do nothing, and it Just Works.

Interesting in trying Trio? You can start with the README , or jump straight to the tutorial. Have fun!

Comments

You can discuss this post on the Trio forum.

---

[1] `KeyboardInterrupt` is an example of the general category of "asynchronous exception". (This is a totally different use of "asynchronous" then the one in "async/await".) If you want to read more about the problems asynchronous exceptions cause, Java made the mistake of including these as a feature in an early release and got stuck with them before realizing how impossible they are to use safely, so they have lots of documentation about their challenges and why they should be avoided.

[2] In fact, this aspect of Unix design is so controversial that it served as the central example of a rather famous essay.

One might also wonder how the kernel actually goes about cancelling a system call. The full answer is a bit complicated, but basically what it comes down to is that when a signal arrives it sets an internal flag, and then when you're implementing a system call inside the kernel you have to remember to check for that flag at appropriate places... Conceptually it's extremely similar to what we end up doing to deliver `KeyboardInterrupt` via trio's cancellation system!

Basically there are only a few different concepts here, and they just get remixed over and over at different parts of the stack :-). If you squint there's even a lot of commonality between trio's extremely-high-level manipulation of Python's stack data to enable/disable `KeyboardInterrupt` for particular stretches of code, and the extremely-low-level concept of enabling or disabling interrupts on a CPU or microcontroller. Ancient x86 CPUs even added hacks to skip processing interrupts during certain instruction sequences used for stack switching, which is a strikingly similar to the way we'll see we need to modify CPython's bytecode loop to skip processing signals during opcodes used in resuming a switching between coroutine stacks.

[3] For those keeping score: `WSASend` wakes up `select` and its variants, `PostQueuedCompletionStatus` wakes up `GetQueuedCompletionStatus` and its variants, `SetEvent` wakes up `WaitForSingleObject` and its variants, and there are a few other calls that can only be interrupted using `QueueUserAPC`.

[4] There is one exception (no pun intended): if `acquire` blocks and then gets interrupted by a signal, then it has some code to explicitly run signal handlers. This is still OK though, because it only does this *before* the lock is acquired. And of course it only works on Unix...

[5] It uses FAST_DISPATCH instead of DISPATCH.

[6] Another interesting design challenge is that `KeyboardInterrupt` is edge-triggered – we deliver it once and then we're done until the user hits control-C again – while cancellation in trio is normally level-triggered – once we start delivering `Cancelled` exceptions, we keep going until the offending code exits the cancelled region. And in trio, it's possible that we attempt to cancel an operation, then we find out later that our cancellation failed, i.e., the operation succeeded anyway. (This is how Windows' IOCP cancellation semantics work, so we're kinda stuck with it.) Together these two things make life a bit difficult, because we need to keep track of whether `KeyboardInterrupt` was delivered.

One part of this was tweaking the internal cancellation APIs a bit so that they could keep track of whether an exception had actually been delivered or not – previously that wasn't needed. This is why in trio's lowest level API for sleeping the callback used to cancel a sleep gets passed a function that raises an exception, instead of an exception to raise directly – it gives us a hook to record whether the exception was actually delivered.

The other tricky thing is – suppose we pick a task to receive the `KeyboardInterrupt` cancellation, and don't find out immediately whether the delivery was successful. This leaves us in a delicate state; basically it's Schrödinger's interrupt. We can't deliver it to anyone else while the first attempt is pending, because it the first attempt then succeeds we'll have delivered the same interrupt twice. But we can't forget about it either, because the attempt might fail. It might even happen that it fails and then the task we picked exits without passing through another cancellation point, and then we might need to pick another task to deliver it to.

We solve this through a sneaky hack: we always pick the "main" task to receive the `KeyboardInterrupt`. (The main task is the first task started, which is the ultimate ancestor of all other user tasks.) This means we don't have to keep track of delivery failures explicitly, because if the main task hits a second checkpoint without the first delivery having succeeded, then it must have failed. And we don't have to worry about switching to a different victim task, because the main task is always the last user task to exit, so if it exits then we can fall back on our logic for a control-C that arrives during shutdown. So this simplification actually solves a rather difficult problem!

[7]
A particularly fun issue we have to work around in the `KeyboardInterrupt` protection decorators is bpo-29590: `throw`ing into a generator or coroutine breaks stack introspection. Obviously this is a problem when the whole idea is for the signal handler to introspect the stack. Most of the time trio works around this by... never ever using the `throw` method. (This is also necessary to avoid hitting bpo-29587. `throw` is really buggy.) But a major use case for `enable_ki_protection` is on context managers, and `contextlib.contextmanager` uses `throw`, so...

Perhaps you can imagine how much fun I had debugging this the first time I ran into it.

Posted by Nathaniel J. Smith Wed 19 April 2017